

MANUAL DE COMANDOS

Trinnity Viseron System v5.0

Comandos Viseron + 8 GitHub Tools Integradas

yt-dlp | Ollama | Fooocus | Whisper | Plausible | AppFlowy | Penpot | CUDACyclone

Gerado: 02/08/2026, 15:23:16

Trinnity Hurtado — Reina (Coroa) | Pedro Costa — Capitan (Hierro)

SUMARIO

1. Comandos Viseron (npm + CLI + API) — lido de package.json
2. tvs_ytdlp — YouTube/Video Downloader
3. tvs_ollama — IA Local (LLaMA, Qwen, Mistral...)
4. tvs_foocus — Gerador de Imagens AI
5. tvs_whisper — Audio para Texto
6. tvs_plausible — Web Analytics
7. tvs_appflowy — Workspace AI
8. tvs_penpot — Design Tool + MCP
9. tvs_cudacyclone — GPU Bitcoin Puzzle Solver (CUDA)
10. API TVS Completa (auth/billing/onboarding/email)
11. Como emitir comandos via Diretivas

1. COMANDOS VISERON

Comandos npm para operar o Trinnity Viseron System (lidos automaticamente de package.json — novos comandos aparecem aqui a cada regeneração):

```
npm run start
Executar sistema compilado (dist)
npm run start:exe
Executar executável standalone Windows
npm run dev
Modo desenvolvimento com hot reload
npm run build
Compilar TypeScript para dist/
npm run test
Rodar testes core + web (67)
npm run test:core
Rodar testes do núcleo
npm run test:web
Rodar testes da camada web (auth/billing/onboarding/email/messaging)
npm run demo
Demo operacional real (HTTP 9 endpoints)
npm run test:hyper
Rodar testes hyperbrain
npm run build:android
Gerar APK Android
npm run build:ios
Gerar IPA iOS
npm run build:all
Gerar APK + IPA
npm run build:eas-android
Build Android via EAS
npm run build:eas-ios
Build iOS via EAS
npm run mobile:start
Iniciar app mobile Expo
npm run mobile:android
Executar app Android
npm run mobile:ios
Executar app iOS
npm run launch
Script de lançamento de mercado
npm run setup
Instalar todas as dependências
npm run lint
Verificar TypeScript (tsc --noEmit)
npm run backup
Executar backup diário
npm run backup:schedule
Agendar backup automático (Task Scheduler)
npm run skills
Skills CLI (list, search, info)
npm run skills:install
Instalar/atualizar coleções de skills (autônomo)
npm run skills:list
Listar skills instaladas
npm run skills:search
Pesquisar skills
npm run skills:info
Ver info de uma skill
npm run init
Build + backup + start
npm run init:full
Inicialização completa do sistema
npm run deploy
Deploy completo (build + PDFs + backup)
npm run deploy:github
Deploy GitHub
npm run deploy:render
Deploy Render via API
```

```

npm run deploy:hostalia
Deploy landing page via FTP Hostalia
npm run deploy:vercel
Deploy Vercel
npm run start:web
Executar servidor web compilado
npm run dev:web
Servidor web em modo desenvolvimento
npm run pitch
Gerar pitch de investidores
npm run pitch:startup
Gerar pitch startup (3 idiomas)
npm run pitch:v6
Gerar pitch v6
npm run roadmap
Gerar roadmap milionário
npm run docs:100
Gerar data/Viseron_100_Melhorias_Integracao.pdf
npm run report:update
Gerar relatório de update PDF
npm run report:state
Gerar relatório de estado PDF (o que podes fazer + estado real)
npm run docs:revenue
Gerar plano de receita real PDF (passo a passo Stripe/Gmail/domínio + modelo de receita)
npm run audit:arkom
Auditoria operacional ARKOM/AIOX (scan/diagnóstico/fix/verdicto !' Viseron_Audit_ARKOM.pdf)
npm run demo:jarvis
Demo do JARVIS (conversa + autonomia real sobre o sistema)
npm run gmail:setup
Setup Gmail API (OAuth consent !' refresh token)
npm run demo:email
Demo dos 3+ fluxos de email (verify/reset/invoice/agent)
npm run demo:messaging
Demo de mensageria E2E (contactos/conversas/grupos/leitura)
npm run pdfs:all
Regenerar TODOS os PDFs (manuais, pitches, roadmap, 100 melhorias)
npm run deploy:domain
Configurar domínio www.trinnityviseronssystem.io no .env
npm run deploy:domain:check
Validar DNS/HTTPS do domínio novo
npm run update:auto
Self-update: pull + install + PDFs + build + testes + deploy
npm run omniroute:install
npm install omniroute
npm run omniroute:start
node -e "const{OmniRouteBridge}=require('./dist/src/integrations/omniroute/OmniRouteBridge');new OmniRouteBridge(null,
{port:20128,autoStart:true}).initialize()"
npm run build:exe
node scripts/build-standalone.mjs
npm run build:exe:win
node scripts/build-standalone.mjs --platform win
npm run build:exe:mac
node scripts/build-standalone.mjs --platform mac
npm run build:exe:linux
node scripts/build-standalone.mjs --platform linux
npm run build:exe:all
node scripts/build-standalone.mjs --platform all
npm run build:electron
node scripts/build-standalone.mjs --platform win --electron
npm run build:electron:all
node scripts/build-standalone.mjs --platform all --electron
npm run electron:setup
cd electron && npm install
npm run electron:start
cd electron && npx electron .
npm run electron:build:win
cd electron && npm run build:win
npm run electron:build:mac
cd electron && npm run build:mac
npm run electron:build:linux
cd electron && npm run build:linux
npm run call:start
node -e "require('./dist/src/integrations/call-system/CallSystemBridge').startServer()"

```

```

npm run jarvis:start
node -e "require('./dist/src/integrations/openjarvis/OpenJarvisBridge').startServer()"
npm run asno:start
node -e "require('./dist/src/integrations/asno/ASNOBridge').startServer()"
npm run super:start
src/index.ts
npm run deploy:all
scripts/deploy-all.ps1 -Full
npm run deploy:full
scripts/deploy-all.ps1 -Full
npm run audit:full
node scripts/full-audit.mjs
npm run cudacyclone
scripts/cudacyclone.ts
npm run domain:check
scripts/setup-dominio.ps1 -Validate
npm run go-live:stripe
scripts/go-live-stripe.ts

```

2. yt-dlp — Downloader Universal

Repositorio: github.com/yt-dlp/yt-dlp

Baixa videos/audios de YouTube, Twitter, TikTok, Instagram, Twitch, Facebook e 1000+ sites.

Instalacao: `pip install yt-dlp`

Comandos Viseron:

```

tvs_ytdlp url='https://youtube.com/watch?v=...' modo=video qualidade=best
!' Baixar video na melhor qualidade
tvs_ytdlp url='...' modo=audio
!' Baixar apenas audio (MP3)
tvs_ytdlp url='...' modo=video qualidade=4k
!' Baixar video em 4K
tvs_ytdlp url='...' modo=video playlist=no
!' Baixar apenas um video (nao playlist)
tvs_ytdlp url='...' legendas=true lang='pt,en'
!' Baixar com legendas
tvs_ytdlp url='...' dir='./meus-videos'
!' Salvar em diretorio especifico

```

3. Ollama — IA Local

Repositorio: github.com/ollama/ollama

Executa modelos de linguagem localmente: LLaMA, Qwen, Mistral, DeepSeek, Phi, Gemma.

Instalacao: ja instalado (ollama v0.32.5)

Comandos Viseron:

```

tvs_ollama acao=listar
!' Listar modelos disponiveis localmente
tvs_ollama acao=puxar modelo='llama3.2'
!' Baixar modelo LLaMA 3.2
tvs_ollama acao=chat modelo='llama3.2' prompt='O que e IA?'
!' Perguntar ao modelo local
tvs_ollama acao=embed modelo='llama3.2' prompt='texto'
!' Gerar embedding de texto
tvs_ollama acao=remove modelo='modelo-antigo'
!' Remover modelo baixado
ollama run deepseek-r1:7b
!' Comando direto: rodar DeepSeek R1

```

4. Fooocus — Gerador de Imagens AI

Repositorio: github.com/lllyasviel/Fooocus

Gera imagens fotorrealistas com IA, estilo Midjourney, sem necessidade de engenharia de prompt.

Instalacao: `git clone https://github.com/lllyasviel/Fooocus.git && pip install -r requirements.txt`

Comandos Viseron:

```

tvs_fooocus acao=gerar prompt='gato astronauta' estilo=fantasy
!' Gerar imagem com descricao
tvs_fooocus acao=gerar prompt='...' negativo='borrado, feio'
!' Gerar com prompt negativo
tvs_fooocus acao=gerar prompt='...' passos=50 largura=1920 altura=1080
!' Imagem HD com mais qualidade
tvs_fooocus acao=status
!' Verificar status do servidor Fooocus
cd ../Fooocus && python entry_with_update.py
!' Iniciar interface web (porta 7865)

```

5. Whisper — Audio para Texto

Repositorio: github.com/openai/whisper

Transcreve audio/video para texto com IA. Suporta 99+ idiomas. Modelos: tiny, base, small, medium, large, turbo.

Instalacao: `pip install openai-whisper`

Comandos Viseron:

```
    tvs_whisper acao=transcrever arquivo='audio.mp3' modelo=medium
!' Transcrever audio em portugues
    tvs_whisper acao=transcrever arquivo='video.mp4' modelo=large idioma=en
!' Transcrever video em ingles
    tvs_whisper acao=transcrever arquivo='aula.wav' modelo=turbo formato=vtt
!' Transcrever e gerar legendas VTT
    tvs_whisper acao=listar
!' Listar modelos disponiveis com descricao
    python -m whisper audio.mp3 --model base --language pt
!' Comando direto Python
```

6. Plausible — Web Analytics

Repositorio: github.com/plausible/community-edition

Analytics web leve, open-source, respeitador de privacidade. Alternativa ao Google Analytics.

Instalacao: `git clone https://github.com/plausible/community-edition.git && docker-compose up -d`

Comandos Viseron:

```
    tvs_plausible acao=deploy
!' Instrucoes para deploy com Docker
    tvs_plausible acao=status
!' Verificar status do servidor Plausible
    tvs_plausible acao=stats site='meusite.com' chave='API_KEY'
!' Obter estatisticas do site
    git clone https://github.com/plausible/community-edition.git
!' Clonar repositorio
    docker-compose up -d
!' Iniciar com Docker (porta 8000)
```

7. AppFlowy — Workspace AI

Repositorio: github.com/AppFlowy-IO/AppFlowy

Workspace colaborativo open-source com IA integrada. Alternativa ao Notion e Monday.com.

Instalacao: `git clone https://github.com/AppFlowy-IO/AppFlowy.git && docker-compose up -d`

Comandos Viseron:

```
    tvs_appflowy acao=deploy
!' Instrucoes para deploy Docker
    tvs_appflowy acao=status
!' Verificar containers AppFlowy rodando
    git clone https://github.com/AppFlowy-IO/AppFlowy.git
!' Clonar repositorio
    docker-compose up -d
!' Iniciar servicos AppFlowy
```

8. Penpot — Design + MCP

Repositorio: github.com/penpot/penpot-mcp

Ferramenta de design open-source com protocolo MCP. Alternativa ao Figma com integracao AI.

Instalacao: `git clone https://github.com/penpot/penpot.git && git clone https://github.com/penpot/penpot-mcp.git`

Comandos Viseron:

```
    tvs_penpot acao=deploy
!' Instrucoes para deploy Penpot + MCP
    tvs_penpot acao=mcp
!' Configuracao do servidor MCP Penpot
    tvs_penpot acao=exportar projeto='ID'
!' Exportar design para SVG/PNG/HTML
    git clone https://github.com/penpot/penpot.git
!' Clonar repositorio principal
    git clone https://github.com/penpot/penpot-mcp.git
!' Clonar repositorio MCP
```

9. CUDACyclone — GPU Bitcoin Puzzle Solver

Repositorio: github.com/Dookoo2/CUDACyclone

Solver de 'Satoshi puzzles' (busca de chave privada em intervalo) com aceleracao CUDA em GPU NVIDIA. Baseado em VanitySearch/Bitcrack. 6.5Gkeys/s (RTX 4090). Requer NVIDIA GPU + CUDA toolkit (Linux/WSL2).

Instalacao: `git clone https://github.com/Dookoo2/CUDACyclone.git && make` (com CUDA toolkit instalado)

Comandos Viseron:

```
    tvs_cudacyclone acao=status
!' Verificar binario, requisitos e instalacao
    tvs_cudacyclone acao=build
!' Clonar e compilar com make (GPU NVIDIA + CUDA)
    tvs_cudacyclone acao=run range='2000000000:3FFFFFFFFF' address='1HBtApAFA9B2YZw3G2YKSMctb3dVnjuNe2' grid='512,256'
!' Buscar chave privada no intervalo
    tvs_cudacyclone acao=run range='...' target-hash160='...' grid='128,128' slices='16'
!' Buscar por hash160 (sem precisar de endereco)
    tvs_cudacyclone acao=benchmark
!' Velocidades por GPU (RTX 4060/4090/5090/3070)
    ./CUDACyclone --range 2000000000:3FFFFFFFFF --address 1HBtApAFA9B2YZw3G2YKSMctb3dVnjuNe2 --grid 512,256
!' Comando direto (RTX 4060 ~1238 Mkeys/s)
    ./CUDACyclone --range 2000000000:3fffffffffff --address 1F3JRMwudBaj48EhwcHDdpeuy2jwACNxjP --grid 128,128 --
slices 16
!' Tune otimizado RTX 4090 (~6.1 Gkeys/s)
```

10. API TVS COMPLETA

Endpoints REST para controlar o TVS remotamente:

Endpoint	Descricao
GET /api/health	Health + db + billing + email + contagens
GET /api/metrics	Métricas de uso
GET /api/system/status	Status do servidor standalone
POST /api/waitlist	Registrar email na waitlist
GET /pitch/:file	Baixar PDFs de pitch
POST /api/auth/register	Registo multi-tenant (org !' tenant + owner + JWT)
POST /api/auth/login	Login JWT (rate-limited)
GET /api/auth/me	Perfil autenticado
PATCH /api/auth/profile	Atualizar nome/perfil/role
GET /api/auth/users	Listar membros (owner/admin)
POST /api/auth/logout	Terminar sessão
GET /api/billing/plans	Planos Core \$29 / Pro \$99 / Enterprise \$499
POST /api/billing/checkout	Criar sessão de checkout
POST /api/billing/webhook	Webhook de pagamento !' upgrade do plano
GET /api/billing/subscription	Estado da subscrição/trial
GET /api/onboarding/templates	5 templates (conteúdo, atendimento, código, Squad AIOX, Arkom)
POST /api/onboarding/apply	Materializar agentes no workspace do tenant
GET /api/email/status	Provider de email ativo + estado Gmail
POST /api/email/test	Enviar email de teste
POST /api/email/verify/send	Enviar código de verificação
POST /api/email/verify/confirm	Confirmar código de verificação
GET /api/email/verified	Estado de verificação do email
POST /api/email/reset/send	Enviar código de reposição de password
POST /api/email/reset/confirm	Confirmar código + nova password
GET /api/messaging/status	Estado da mensageria + crypto (x25519/aes-256-gcm)
POST /api/messaging/key	Gerar/obter chave pública X25519 do utilizador
GET /api/messaging/contacts	Listar contactos
POST /api/messaging/contacts	Adicionar contacto por email
GET /api/messaging/conversations	Listar conversas + unread
POST /api/messaging/conversations	Criar conversa direta
POST /api/messaging/groups	Criar grupo
GET /api/messaging/conversations/:id/messages	Ler mensagens (desencriptadas para o membro)
POST /api/messaging/conversations/:id/messages	Enviar mensagem E2E (cifrada por recetor)
POST /api/messaging/conversations/:id/read	Marcar conversa como lida
GET /api/blog/posts	Listar posts do blog
POST /api/content/generate	Gerar post de conteúdo
POST /api/content/trigger	Disparar geração de conteúdo
GET /api/content/schedule	Estado do agendamento de conteúdo
GET /api/jarvis/status	Estado do JARVIS (provider, capacidades, autonomia)
POST /api/jarvis/chat	Conversar com o JARVIS (sessão + execução real de operações,
rate-limited 30/min)	
GET /api/revenue/readiness	Go-live de receita real: Stripe, webhook, Gmail, email provider,
domínio, Postgres	

11. COMO EMITIR COMANDOS VIA DIRETIVAS

Toda ferramenta no TVS é acessível via diretivas. O fluxo é:

1. Agente decide qual ferramenta usar baseado na missao
2. Agente chama toolManager.executeTool('tvs_ytdlp', { url: '...' })
3. ToolManager executa o comando real no sistema
4. Resultado retorna ao agente com sucesso/erro + dados
5. Agente consolida resposta e retorna ao usuario

Exemplo de Diretiva:

```
POST /api/directive
{
  "id": "directive_yt_001",
  "objective": "Baixar video do YouTube sobre IA",
  "squad": ["agent_mind_343_elon_musk"],
  "ratifiedBy": "trinnity-hurtado",
  "commandedBy": "pedro-costa",
  "budget": "100 VSR"
}
```

Exemplo de Chat Direto:

```
Agente: "Elon, baixa o ultimo video do Neuralink"
Elon: "tvs_ytdlp url=https://youtube.com/... modo=video"
Resultado: Video baixado em ./data/downloads/
```


